

TROUBLESHOOTING

Troubleshooting

Fix high CPU or memory usage, hangs, auto-compact thrashing, and search problems in Claude Code, and find the right page for other issues.

This page covers performance, stability, and search problems once Claude Code is running. For other issues, start with the page that matches where you’re stuck:

Symptom	Go to
<code>command not found</code> , install fails, PATH issues, <code>EACCES</code> , TLS errors	<u>Troubleshoot installation and login</u>
Login loops, OAuth errors, <code>403 Forbidden</code> , “organization disabled”, Bedrock/Vertex/Foundry credentials	<u>Troubleshoot installation and login</u>
Settings not applying, hooks not firing, MCP servers not loading	<u>Debug your configuration</u>
<code>API Error: 5xx</code> , <code>529 Overloaded</code> , <code>429</code> , request validation errors	<u>Error reference</u>
<code>model not found</code> or <code>you may not have access to it</code>	<u>Error reference</u>
VS Code extension not connecting or detecting Claude	<u>VS Code integration</u>
JetBrains plugin or IDE not detected	<u>JetBrains integration</u>
High CPU or memory, slow responses, hangs, search not finding files	<u>Performance and stability</u> below

If you’re not sure which applies, run `/doctor` inside Claude Code for an automated check of your installation, settings, MCP servers, and context usage. If `claude` won’t start at all, run `claude doctor` from your shell instead.

Performance and stability

These sections cover issues related to resource usage, responsiveness, and search behavior.

High CPU or memory usage

Claude Code is designed to work with most development environments, but may consume significant resources when processing large codebases. If you're experiencing performance issues:

1. Use `/compact` regularly to reduce context size
2. Close and restart Claude Code between major tasks
3. Consider adding large build directories to your `.gitignore` file
4. Restart with `claude --safe-mode` to check whether a plugin, MCP server, or hook is the source. It disables all customizations for the session; if usage drops, see [Debug your configuration](#) to find which one

If memory usage stays high after these steps, run `/heapdump` to write a JavaScript heap snapshot and a memory breakdown to `~/Desktop`. On Linux without a Desktop folder, the files are written to your home directory.

The breakdown shows resident set size, JS heap, array buffers, and unaccounted native memory, which helps identify whether the growth is in JavaScript objects or in native code. To inspect retainers, open the `.heapsnapshot` file in Chrome DevTools under Memory → Load. Attach both files when reporting a memory issue on [GitHub](#).

Auto-compaction stops with a thrashing error

If you see `Autocompact is thrashing: the context refilled to the limit...`, automatic compaction succeeded but a file or tool output immediately refilled the context window several times in a row. Claude Code stops retrying to avoid wasting API calls on a loop that isn't making progress.

To recover:

1. Ask Claude to read the oversized file in smaller chunks, such as a specific line range or function, instead of the whole file
2. Run `/compact` with a focus that drops the large output, for example `/compact keep only the plan and the diff`
3. Move the large-file work to a [subagent](#) so it runs in a separate context window
4. Run `/clear` if the earlier conversation is no longer needed

Command hangs or freezes

If Claude Code seems unresponsive:

1. Press Ctrl+C to attempt to cancel the current operation
2. If unresponsive, you may need to close the terminal and restart

Restarting doesn't lose your conversation. Run `claude --resume` in the same directory to pick the session back up.

Garbled or corrupted text in an editor's integrated terminal

If characters render as boxes, smears, or the wrong glyphs when running Claude Code in the VS Code, Cursor, or Devin Desktop integrated terminal, the terminal's GPU renderer is likely the cause. Run `/terminal-setup` inside Claude Code to set `terminal.integrated.gpuAcceleration` to `"off"`, or set it manually in your editor settings and reload the window. See [Terminal configuration](#) for the other settings `/terminal-setup` writes.

Search and discovery issues

If the Search tool, `@file` mentions, custom agents, or custom skills aren't finding files, the bundled `ripgrep` binary may not run on your system. Install your platform's `ripgrep` package and tell Claude Code to use it instead:

macOS

```
brew install ripgrep
```

Ubuntu/Debian

```
sudo apt install ripgrep
```

Alpine

```
apk add ripgrep
```

Arch

```
pacman -S ripgrep
```

Windows

```
winget install BurntSushi.ripgrep.MSVC
```

Then set `USE_BUILTIN_RIPGREP=0` in your environment.

Slow or incomplete search results on WSL

Disk read performance penalties when working across file systems on WSL may result in fewer-than-expected matches when using Claude Code on WSL. Search still functions, but returns fewer results than on a native filesystem.

ⓘ `/doctor` will show Search as OK in this case.

Solutions:

1. **Submit more specific searches:** reduce the number of files searched by specifying directories or file types: “Search for JWT validation logic in the auth-service package” or “Find use of md5 hash in JS files”.
2. **Move project to Linux filesystem:** if possible, ensure your project is located on the Linux

filesystem (`/home/`) rather than the Windows filesystem (`/mnt/c/`).

3. **Use native Windows instead:** consider running Claude Code natively on Windows instead of through WSL, for better file system performance.

Get more help

If you're experiencing issues not covered here:

1. Run `/doctor` to check installation health, settings validity, MCP configuration, and context usage in one pass
2. Use the `/feedback` command within Claude Code to report problems directly to Anthropic
3. Check the [GitHub repository](#) for known issues
4. Ask Claude directly about its capabilities and features. Claude has built-in access to its documentation.